

Inkludera filer

Den viktigaste uppgiften som preprocessorn utför är *inkludering av filer*. Inkludering av filer sköts med hjälp av `#include`. Om man tittar på en lista av nyckelord som används i C++ finns ordet `include` inte med. Orsaken är enkel: C++-kompilatorn ser aldrig ordet `include` i den text som den kompilerar, preprocessorn har redan ersatt `#include`-definitionen med innehållet i korrekt fil.

Man använder alltså `#include` för att inkludera definitioner från andra filer. Hittills har vi använt `#include` för att inkludera information om bl.a. *streams* (`cin` och `cout`) och strängar. Vad vi egentligen gjort är att vi instruerat preprocessorn att inkludera stora mängder definitioner för olika datatyper och funktioner. Vi kan sedan i vårt program använda alla de definitioner som finns i de inkluderade filerna. Allmänt ser en `include`-deklaration ut så här:

```
#include "filnamn"
#include <filnamn>
```

Notera att man inte behöver något semikolon (;) efter definitionen, eftersom det inte är frågan om en normal C++-instruktion. De två olika formerna av `#include` fungerar på olika sätt. Skillnaden ligger i var preprocessorn försöker hitta filerna. Den första definitionen kommer att försöka hitta *filnamn* i den aktuella arbetskatalogen först, och om filen inte hittades där, så genomöks andra kataloger som antingen givits till kompilatorn eller som är kompilerspecifika. Den andra definitionen gör att preprocessorn inte ens försöker hitta filen i aktuell arbetskatalog, utan direkt söker i de systemspecifika katalogerna (t.ex. under `/usr/include` på Unix). Den första formen används normalt om man har ett eget program som använder en definitionsfil. Några exempel på `#include`-definitioner:

```
#include <iostream>
#include <string>
#include <stdio.h>
#include "MinaDefinitioner.h"
#include "Konstanter"
```

Man brukar placera alla `#include`-deklarationer i början av programmen, eftersom inga funktioner som definieras i en header-fil kan användas innan de definierats, d.v.s. innan de inkluderats. Traditionellt har man gett header-filer extensionen `.h`, men i och med nya standarder för C++ är detta inte längre nödvändigt. Därför har många av de `#include`-definitioner vi använder ingen `.h`-extension. Mera information om header-filer och externa bibliotek finns i [Kapitel 11](#).

Standarden för C++

Standarden för C++ som relativt nyligen har fått en relativt stabil form ändrar på hur header-filer skall namnges. Enligt denna standard skall *alla* header-filer vara utan extensionen `.h`. Vi har ju redan sett att alla C++-specifika header-filer redan saknar denna extension, men gamla C-specifika header-filer ännu använder sig av extensionen. Dessa skall istället få ett prefix `c` och sedan även lämna bort extensionen `.h`. Vi får då att t.ex. `stdio.h` och `string.h` blir enligt standarden `cstdio` och `cstring`.

Det kan dock ta en tid innan alla kompilatorer fullt gått in för denna namngivning av header-filer,

främst för att det finns enorma mängder gamla C-program som är beroende av den gamla namngivningen. På grund av detta kommer de två sätten att finnas parallellt ännu en lång tid, kanske för alltid. Det rekommenderas dock att man använder sig av den nya namngivningen om kompilatorn understöder detta, även om programmet i detta kompendium inte gör det.

[Förutgående](#)
Preprocessorn

[Hem](#)
[Upp](#)

[Nästa](#)
Konstanter